# 1. Introduction

Dans le cadre du développement de la plateforme SaaS fintech **Quantara**, ce document définit l'architecture technique du backend. Après avoir établi la vision globale et le périmètre du Produit Minimum Viable (MVP), il est impératif de verrouiller les choix technologiques et structurels qui garantiront la robustesse et la pérennité du système.

L'objectif est de bâtir une architecture **propre, modulaire, maintenable et extensible**, capable de supporter la charge du MVP tout en étant prête pour les évolutions futures. Une structuration rigoureuse dès le départ permet d'éviter les écueils classiques tels que la dette technique précoce, le code désorganisé ou les difficultés de testabilité.

# 2. Objectif de l'Architecture Backend

L'architecture backend de Quantara est conçue pour offrir une base technique saine capable de supporter les piliers fonctionnels du produit : l'authentification, la gestion des données de marché, le moteur de simulation, les modules d'éducation et le suivi de progression.

En structurant le système dès sa genèse, nous visons à éliminer les problèmes suivants :

- La logique métier mélangée aux routes API.
- Les endpoints surchargés et difficiles à maintenir.

- La présence de requêtes SQL directes dans les couches de présentation.
- Les services “fourre-tout” manquant de cohésion.
- Le code difficilement testable de manière isolée.

> ***L’ambition est de construire une architecture simple pour le démarrage, mais suffisamment bien pensée pour ne pas nécessiter de refonte majeure lors de la montée en charge.***

# 3. Stack Technique Retenue

Le choix des technologies repose sur des critères de performance, de robustesse, de maintenabilité et de rapidité de développement.

## FastAPI

FastAPI est le framework principal du backend. Il a été choisi pour sa rapidité d’exécution, sa facilité de mise en place et sa gestion native de l’asynchronisme. Il offre une validation forte des données via Pydantic et génère automatiquement une documentation interactive (Swagger/OpenAPI), ce qui est essentiel pour la collaboration avec l’équipe frontend et la clarté des contrats d’API.

## PostgreSQL

PostgreSQL est la base de données relationnelle principale. Sa robustesse, sa fiabilité et sa richesse fonctionnelle en font le choix idéal pour stocker les données critiques et persistantes de la plateforme : profils utilisateurs, états des portefeuilles, historiques de transactions et progression pédagogique.

## SQLAlchemy & Alembic

SQLAlchemy est utilisé comme ORM (Object-Relational Mapping) pour manipuler les entités de manière propre et pythonique, en faisant abstraction des détails SQL. Alembic complète cet outil en gérant les migrations de base de données, permettant

de versionner le schéma et de le faire évoluer de manière contrôlée sans rupture de service.

### Pydantic

Pydantic assure la validation des données entrantes et la définition des schémas de réponse. Il garantit que les contrats d'interface entre le frontend et le backend sont strictement respectés, améliorant ainsi la fiabilité et la sécurité des échanges de données.

## 4. Architecture en Couches

Le backend de Quantara adopte une architecture en couches claires, favorisant la séparation des préoccupations et la modularité. Chaque couche a des responsabilités bien définies :

### Couche 1 — API

Cette couche est le point d'entrée des requêtes HTTP. Ses responsabilités incluent le routage des requêtes, le parsing des données entrantes, la validation initiale des entrées (via Pydantic), l'appel du service métier approprié et le formatage des réponses. Elle ne doit pas contenir de logique métier complexe.

### Couche 2 — Service Métier

C'est le cœur fonctionnel de l'application. Cette couche est responsable d'appliquer les règles métier, de coordonner les interactions entre plusieurs repositories, d'appeler les fournisseurs externes si nécessaire, et de prendre les décisions métier. C'est ici que réside la logique principale du produit.

## Couche 3 — Repository

Cette couche gère exclusivement l'accès à la base de données. Ses responsabilités comprennent l'exécution des requêtes DB (lecture, écriture, mise à jour, suppression), la gestion des filtres simples et des opérations CRUD (Create, Read, Update, Delete) via SQLAlchemy. Le repository ne contient aucune logique métier et se concentre uniquement sur la persistance des données.

## Couche 4 — Modèles & Schémas

Cette couche décrit la structure des données. Elle contient les modèles SQLAlchemy qui définissent les entités persistées en base de données, ainsi que les schémas Pydantic qui décrivent les structures de données échangées via l'API (modèles de requête et de réponse).

## Couche 5 — Infrastructure / Providers

Cette couche gère les interactions avec les systèmes externes et les services techniques. Cela inclut les fournisseurs de données externes (ex: CoinGecko), les mécanismes de cache (à terme), les connexions externes et les adaptations techniques spécifiques à l'environnement.

**Schéma Logique Simplifié du Flux Backend :**

```
Requête HTTP
      ↓
  API Route
      ↓
   Service
      ↓
Repository / Provider / Rules
      ↓
Database or External Source
      ↓
   Réponse
```

Ce flux doit rester simple et stable à travers tous les modules, garantissant une compréhension claire du cheminement des données et de la logique.

# 5. Structure Globale du Projet

L'arborescence du projet backend est conçue pour refléter l'architecture modulaire et en couches, facilitant la navigation, la maintenance et l'évolutivité.

```
backend/
├── pyproject.toml      # Configuration du projet Python (dépendances,
outils)
├── requirements.txt    # Liste des dépendances Python
├── alembic.ini         # Configuration d'Alembic pour les migrations de base
de données
├── app/
│   ├── main.py         # Point d'entrée principal de l'application FastAPI
│   ├── core/           # Configurations globales (settings, sécurité,
logging, exceptions)
│   ├── db/             # Configuration de la base de données, session
SQLAlchemy, migrations
│   ├── api/            # Définition des routes API versionnées (v1/) et
dépendances
│   ├── modules/        # Cœur modulaire du backend, découpé par domaine
métier
│   ├── shared/         # Code partagé transversal (enums, utilitaires,
types)
│   └── tests/          # Tests unitaires et d'intégration
└── scripts/            # Scripts techniques utiles (seed data,
initialisation locale)
```

# 6. Rôle des Principaux Dossiers

### `app/main.py`

Ce fichier est le point d'entrée principal de l'application FastAPI. Il est responsable de la création de l'instance de l'application, de l'enregistrement des routes, de la configuration des middlewares (CORS, gestion des erreurs) et du lancement du serveur. Il ne doit pas contenir de logique métier.

### `app/core/`

Ce dossier regroupe les configurations globales et les composants transversaux essentiels à l'application. Il contient typiquement `config.py` (paramètres de l'application), `security.py` (fonctions de sécurité) `logging.py` (configuration des logs) et `exceptions.py` (gestion des exceptions globales).

### `app/db/`

Ce dossier est dédié à la gestion de la base de données. Il contient `base.py` (définition de la base SQLAlchemy), `session.py` (gestion des sessions de base de données) et le dossier `migrations/` (fichiers de migration gérés par Alembic).

### `app/api/`

Ce dossier contient la définition des routes API. Il inclut `deps.py` (dépendances partagées pour les routes, ex: injection de session DB), et les sous-dossiers versionnés comme `v1/` qui contiennent les routeurs spécifiques à chaque version de l'API.

### `app/modules/`

C'est le cœur modulaire du backend, où la logique métier est découpée en domaines fonctionnels autonomes. Chaque sous-dossier représente un module métier (ex: `identity`, `market_data`, `simulation`). Chaque module doit rester autonome et lisible.

### `app/shared/`

Ce dossier contient le code partagé transversal qui n'appartient pas spécifiquement à un module métier. Exemples : enums, types de données génériques, fonctions utilitaires, modèles de réponses standardisées. Il est crucial de veiller à ce que ce dossier ne devienne pas un fourre-tout, mais qu'il contienne uniquement des éléments véritablement transversaux et réutilisables.

### `app/tests/`

Ce dossier contient l'ensemble des tests pour le backend, organisés par type (unitaires, intégration) et par module. Il est essentiel pour garantir la qualité et la fiabilité du code.

### `scripts/`

Ce dossier regroupe les scripts techniques utiles pour le développement, le déploiement ou l'administration (ex: scripts de *seeding* de données, d'initialisation locale, d'importation de données de test).

# 7. Architecture Modulaire Interne

Chaque module au sein du dossier `app/modules/` suit une structure interne standardisée, garantissant une cohérence et une prévisibilité à travers l'ensemble du backend. Prenons l'exemple du module `simulation` :

```
modules/simulation/
├── models.py      # Modèles SQLAlchemy spécifiques à la simulation
(Portfolio, Position, Trade)
├── schemas.py     # Schémas Pydantic pour les requêtes et réponses API de
simulation
├── repository.py  # Logique d'accès aux données pour la simulation (CRUD
sur Portfolio, Position, Trade)
├── service.py     # Logique métier de la simulation (exécution de trades,
calcul de P&L)
└── rules.py       # Règles métier pures et calculs spécifiques à la
simulation
```

# 8. Rôle des Fichiers Standard dans un Module

### `models.py`

Ce fichier définit les entités persistées en base de données pour le module. Il contient les définitions des tables SQLAlchemy, leurs colonnes, leurs relations et leurs contraintes. Par exemple, dans le module `simulation`, on trouverait les modèles `Portfolio`, `Position`, `Trade` et `TransactionHistory`.

### `schemas.py`

Ce fichier définit les contrats d'entrée et de sortie des APIs pour le module, en utilisant Pydantic. Il contient les modèles de requête (pour la validation des données reçues) et les modèles de réponse (pour la sérialisation des données envoyées). Par exemple, `CreateTradeRequest` ou `PortfolioResponse`.

### `repository.py`

Ce fichier encapsule la logique d'accès aux données pour le module. Il contient les fonctions qui interagissent directement avec la base de données via SQLAlchemy pour effectuer des opérations CRUD (Create, Read, Update, Delete) et des requêtes spécifiques. Il ne contient aucune logique métier et se concentre uniquement sur la persistance des données. Exemples : `get_user_by_email()`, `list_assets()`, `get_open_positions()`.

### `service.py`

Ce fichier contient la logique métier principale du module. Il orchestre les opérations en appelant un ou plusieurs repositories, en appliquant les règles métier définies dans `rules.py`, et en coordonnant les interactions avec d'autres services ou fournisseurs externes. Exemples : `login_user()`, `execute_trade()`, `calculate_portfolio_summary()`.

### `rules.py`

Ce fichier regroupe les règles métier pures, les calculs spécifiques et les validations complexes qui peuvent être réutilisées par le service. Il est conçu pour être indépendant de l'infrastructure et de l'accès aux données, facilitant ainsi les tests unitaires. Exemples : `calculate_units()`, `validate_trade_amount()`, `compute_pnl()`.

# 9. Architecture Spécifique des Modules Quantara

Chaque module métier de Quantara est conçu pour encapsuler un domaine fonctionnel spécifique, en respectant la structure interne décrite précédemment.

## A. `identity`

**Responsabilités :** Gestion des utilisateurs, authentification, autorisation et sécurisation des accès. **Contenu probable :** Modèle `User`, service de connexion, fonctions de hachage de mots de passe, gestion des tokens (JWT).

## B. `market_data`

**Responsabilités :** Récupération, normalisation et distribution des données de marché (cryptomonnaies au MVP) depuis des fournisseurs externes. **Contenu probable :** Modèle`Asset` (si persistant), service de récupération des données, repository pour le cache, et adaptateurs pour les `providers/` externes (ex: CoinGecko).

## C. `charting`

**Responsabilités :** Préparation et service des données nécessaires à l'affichage des graphiques. Au MVP, ce module reste léger, se concentrant sur la fourniture de données brutes ou légèrement agrégées.

## D. `simulation`

**Responsabilités :** Exécution des ordres de trading simulés (achat/vente), gestion des positions, mise à jour du portefeuille virtuel, enregistrement de l'historique et calcul des profits/pertes.

## E. `education`

**Responsabilités :** Gestion des cours, des leçons et de la structure pédagogique de la plateforme.

### F. `progress`

**Responsabilités :** Suivi de la progression des utilisateurs dans les cours, agrégation de statistiques simples et préparation des données pour la page de progression.

### G. `insights`

**Responsabilités :** Génération de recommandations, détection d'erreurs fréquentes et implémentation de logique avancée. Au MVP, ce module peut être minimal ou vide, son architecture étant prévue pour une évolution future.

## 10. Versionnement de l'API

Pour garantir la flexibilité et la compatibilité ascendante, l'API du backend sera versionnée dès le début. La structure recommandée est `app/api/v1/`.

- **Avantages :** Permet de faire évoluer l'API sans impacter les clients existants (notamment le frontend), assure une base professionnelle pour la gestion des changements et facilite la maintenance à long terme.
- **Exemples de routes :** `/api/v1/auth/login`, `/api/v1/assets`, `/api/v1/portfolio`.

## 11. Séparation Frontend / Backend / Provider

Un principe architectural fondamental est la stricte séparation des responsabilités entre le frontend, le backend et les fournisseurs de services externes. Le frontend ne doit **jamais** interagir directement avec un fournisseur externe.

Le flux correct est le suivant :

```
Frontend
     ↓
Quantara Backend
     ↓
External Provider
```

Cette séparation est cruciale pour :

- **Sécurité :** Protéger les clés d'API et les informations sensibles des fournisseurs externes.
- **Contrôle :** Permettre au backend de contrôler l'accès, de normaliser les données et d'appliquer des règles métier avant de les exposer au frontend.
- **Normalisation :** Uniformiser les données provenant de différentes sources externes.
- **Performance :** Offrir la possibilité d'implémenter des mécanismes de cache au niveau du backend.
- **Indépendance :** Rendre le frontend indépendant des changements d'API des fournisseurs externes.

# 12. Sécurité et Authentification (Vision Initiale)

Même si le MVP peut commencer avec une authentification simplifiée, l'architecture du backend doit anticiper une solution d'authentification robuste. Cela inclut :

- La définition d'une entité `User` complète en base de données.
- L'implémentation d'un hachage sécurisé des mots de passe.
- Un flux de connexion (login) standard.
- La protection des routes API nécessitant une authentification.
- La gestion des tokens d'accès (ex: JWT) pour les sessions utilisateur.

L'architecture est conçue pour permettre une évolution vers des systèmes d'authentification plus complexes (MFA, SSO) sans nécessiter de refonte majeure.

# 13. Gestion des Données : Philosophie

La base de données PostgreSQL est considérée comme le référentiel de vérité pour les données propres à l'utilisateur et à la plateforme. Cela inclut :

- **Données persistées dès le début :** `users`, `portfolio state`, `trades`, `progress`, `course completion`.

En revanche, les données de marché externes ne seront pas toutes persistées immédiatement. Certaines pourront être récupérées dynamiquement via les fournisseurs externes et mises en cache temporairement au niveau du backend, sans être stockées de manière permanente dans la base de données principale. Cette approche optimise les performances et réduit la charge de stockage.

# 14. Tests

La testabilité est un pilier de cette architecture. Le backend sera conçu pour faciliter l'écriture de tests à différents niveaux :

## Tests Unitaires

Ils se concentrent sur la validation de petites unités de code isolées, telles que les fonctions de calcul, les règles métier (`rules.py`) et les méthodes des services (`service.py`). Ils garantissent la correction logique des composants individuels.

## Tests d'Intégration

Ils vérifient l'interaction correcte entre plusieurs composants, comme les endpoints API, les services et les repositories, y compris les interactions avec la base de données. Ils valident les flux de travail complets et la cohérence du système.

L'architecture permet d'écrire ces tests dès le début du projet, même si l'implémentation complète de la suite de tests peut être progressive.

# 15. Logging et Gestion des Erreurs

Une gestion robuste des logs et des erreurs est essentielle pour le débogage, la surveillance et la maintenance de l'application en production. L'architecture prévoit :

- La définition d'exceptions applicatives claires et hiérarchisées.
- La gestion des erreurs HTTP standardisées.
- La mise en place d'un système de journalisation (logging) de base pour enregistrer les événements importants et les erreurs.

Bien qu'un système de monitoring complexe ne soit pas requis au MVP, les fondations seront posées pour une intégration future.

# 16. Vision Technique Synthétique

En résumé, l'architecture technique du backend de Quantara est conçue pour être :

- **Modulaire :** Découpée en domaines fonctionnels autonomes.
- **Versionnée :** Pour une évolution contrôlée de l'API.
- **Orientée Métier :** La logique métier est au cœur des services.
- **Basée sur des Couches Claires :** Séparation stricte des préoccupations.
- **Indépendante du Frontend :** Via des contrats API bien définis.
- **Extensible :** Facile à faire évoluer avec de nouvelles fonctionnalités.
- **Testable :** Conçue pour faciliter l'écriture de tests unitaires et d'intégration.
- **Robuste :** Fondée sur des technologies éprouvées et des bonnes pratiques.

# 17. Conclusion

Cette architecture technique pose les fondations nécessaires à la réussite de Quantara. En privilégiant une approche modulaire et une séparation claire des responsabilités, nous garantissons un développement fluide pour le MVP et une grande agilité pour les phases ultérieures. La prochaine étape consistera à définir l'organisation exacte des fichiers et à entamer l'implémentation des premières routes API.